

C05 -AT3 -CODE CHALLENGING

Q1 Static Object Counter

20 marks

Create a static data member in Employee to count all Employee objects created. The count must be shared across objects and accessible without maintaining a separate counter in each object.

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Employee {
4     static int count;
5 public:
6     Employee() {
7         count++;
8     }
9     static void show() {
10         cout << "Total Employees = " << count;
11     }
12 };
13 int Employee::count = 0;
14 int main() {
15     int n;
16     cin >> n;
17     for (int i = 0; i < n; i++) {
18         Employee e;
19     }
20     Employee::show();
21     return 0;
22 }
```

Input

5

Output



Visible Test Cases

Test Case 1

Q2 Deep Copy of Student Record

20 marks

Debug a class containing dynamically allocated marks. Implement a deep-copy constructor and destructor. Changing one object's marks must not change the copied object.

```
class Student {
int* marks;
public:
Student(int m) { marks = new int(m); }
Student(const Student& s) {
marks = s.marks; // Bug: shallow copy
}
void setMarks(int m) { *marks = m; }
void display() { cout << *marks << endl; }
};
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Student {
4     int* marks;
5 public:
6     Student(int m) {
7         marks = new int(m);
8     }
9     Student(const Student& s) {
10         marks = new int(*s.marks);
11     }
12     ~Student() {
13         delete marks;
14     }
15     void setMarks(int m) {
16         *marks = m;
17     }
18     void display() {
19         cout << *marks << endl;
20     }
21 };
22 int main() {
23     int m1, m2;
24     cin >> m1 >> m2;
25     Student s1(m1);
26     Student s2 = s1;
27     s2.setMarks(m2);
```

Input

80 95

Output



Visible Test Cases

Test Case 1

Q3 Virtual Destructor Debugging

20 marks

Debug the following inheritance program. A derived object is deleted through a base pointer. Modify the base class so that both destructors execute safely and in the correct order.

```
class Parent {
public:
    ~Parent() { cout << "Parent Destructor" << endl; }
};
class Child : public Parent {
public:
    ~Child() { cout << "Child Destructor" << endl; }
};
int main() {
    Parent* p = new Child();
    delete p;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Parent {
4 public:
5     virtual ~Parent() {
6         cout << "Parent Destructor" << endl;
7     }
8 };
9 class Child : public Parent {
10 public:
11     ~Child() {
12         cout << "Child Destructor" << endl;
13     }
14 };
15 int main() {
16     Parent* p = new Child();
17     delete p;
18     return 0;
19 }
```

Input

Child
Parent

Output



Visible Test Cases

Test Case 1

Q4 Multimedia Player Abstraction

20 marks

Create an abstract MediaPlayer class with a pure virtual play(). Implement MP3Player, WAVPlayer, MP4Player, and AVIPlayer. Select the correct object from the input and invoke play() polymorphically.

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class MediaPlayer {
4 public:
5     virtual void play() = 0;
6 };
7 class MP3Player : public MediaPlayer {
8 public:
9     void play() {
10         cout << "Playing MP3";
11     }
12 };
13 class WAVPlayer : public MediaPlayer {
14 public:
15     void play() {
16         cout << "Playing WAV";
17     }
18 };
19 class MP4Player : public MediaPlayer {
20 public:
21     void play() {
22         cout << "Playing MP4";
23     }
24 };
```

Input

3

Output



Visible Test Cases

Test Case 1

Q5 Animal Sound Debugging

20 marks

Debug the Animal hierarchy so that Dog, Cat, and Cow override sound(). The correct derived sound must be produced through an Animal pointer.

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Animal {
4 public:
5     virtual void sound() {
6         cout << "Animal Sound" << endl;
7     }
8 };
9 class Dog : public Animal {
10 public:
11     void sound() override {
12         cout << "Dog: Bark" << endl;
13     }
14 };
15 class Cat : public Animal {
16 public:
17     void sound() override {
18         cout << "Cat: Meow" << endl;
19     }
20 };
21 class Cow : public Animal {
22 public:
23     void sound() override {
24         cout << "Cow: Moo" << endl;
25     }
26 };
27 int main() {
```

Input

2

Output**Visible Test Cases**

Test Case 1